E K - 1
Staj Başvuru Formu

ÖĞRENCİNİN

T.C. Kimlik No	:	29195388588
Adı Soyadı	:	Yiğit Alp Bilgin
Öğrenci No	:	820210312
Bölümü	:	Bilgisayar Mühendisliği (KKTC)
Cep Tel ve E-posta Adresi	:	90-505-2221450- bilginy21@itu.edu.tr
İkametgah adresi	:	
Sürekli Yazışma Adresi	:	

STAJ YAPILAN İŞYERİNİN

Mersis Numarası	:	
Vergi Kimlik Numarası	:	4780028965
Unvanı	:	INDEKS Bilgisayar A.Ş.
Hizmet Alanı	:	
Telefon No ve E-posta Adresi	:	+90-212-3312121 - info@index.com.tr
Staj Adresi	:	Ayazağa Mahallesi Mimar Sinan Sokak No: 21 Seba Office
	:	Boulevard, D Blok Kat: 1 Ofis No: 11 Sarıyer, 34485 İstanbul
Çalışan Sayısı	:	186
Banka Adı	:	Türkiye Garanti Bankası A.Ş.
Banka Şube Adı ve Kodu	:	Garanti BBVA Topkapı Sanayi Şubesi - 286
IBAN	:	TR360006200038100006200833

STAJIN

Türü	:	Yazılım		
Başlangıç ve Bitiş Tarihleri	:	17.07.2023 - 11.08.2023		
Çalışılan Gün Sayısı	:	20		
Staj Günleri	:	Pazartesi	Salı	Çarşamba
		Perşembe	Cuma	
		Cumartesi	Pazar	

Yukarıda bilgileri yer alan firmada öğrencinin staj yapması uygundur.

O N A Y L A R

Staj Yapılacak Firma Adı

INDEKS Bilgisayar A.Ş.

Tarih 10.07.2023

KAŞE/İmza

İNDEKS BİLGİSAYAR SİSTEMLERİ MÜHENDİSLİK SAN. TİC. A.Ş.
Ayazağa Mah. Mimar Sinan Sok. D Blk.
No:21 D: 11 Sarıyer 34485 İstanbul
Büyük Mükellefler V.D. 4780028965
Sicil No: 256376
Tel: 0212 289 95 25

Bölüm Staj Komisyonu Başkanı

Arş. Gör. Altay ÜNAL

Tarih 10.07.2023

İmza

8- İşletme yetkilileri, mazeretsiz olarak üç (3) iş günü iş yeri stajına gelmeyen öğrenciyi, en geç beş (5) iş günü içinde Fakülte Dekanlığına yazılı olarak bildirir.

9- Öğrencilerin işletmelerde disiplin soruşturmasını gerektirecek davranışlarda bulunmaları halinde, bu durum işletme tarafından Fakülte Dekanlığına yazılı olarak bildirilir. Disiplin işlemi, Fakülte Dekanlığınca Yükseköğretim Kurumları Öğrenci Disiplin Yönetmeliği hükümlerine göre yürütülür. Sonuç, işletmeye yazılı olarak bildirilir.

10- Öğrencinin staj sürecinde;

- Meydana gelebilecek iş kazaları ve meslek hastalıklarından ve çalışmalarına yönelik doğan sigorta prim ödemelerinden eğitim gördüğü kurum,
- Yaşanan iş kazalarının, mesleki hastalıklarının ve varsa geçici iş göremezlik raporlarının bir gün içerisinde işveren statüsündeki eğitim birimine bildirilmesinden staj yapılan firma ve yetkilileri sorumludur.

11- 3308 sayılı Mesleki Eğitim Kanunun 25. Maddesi uyarınca öğrencilere stajları süresince asgari ücretin net tutarının yüzde otuzundan daha düşük ücret ödenemez. Staj yapacak işletme bulunamaması nedeniyle stajını yükseköğretim kurumları ve birimlerinde yapan yükseköğretim öğrencilerinin yaptıkları stajlar bu fıkra hükmü kapsamı dışındadır. Kamu kurum ve kuruluşlarında yapılan stajlarda stajın yapıldığı kuruma Devlet Katkısı ödemesi yapılmaz.

12- Mesleki eğitim görülen işletmede; yirmiden az personel çalışıyor ise ödenebilecek en az ücretin üçte ikisi, yirmi ve üzerinde personel çalışıyor ise ödenebilecek en az ücretin üçte biri, 4447 sayılı Kanunun 53 üncü maddesinin üçüncü fıkrasının (B) bendinin (b) alt bendi için ayrılan tutardan Devlet katkısı olarak staj yapılan işyerine Üniversite tarafından ödenir.

13- Staj yapılan firmaya devlet katkısı ödemelerinin yapılabilmesi için; Firmanın

- Mesleki Eğitim Gören Öğrencilere Yapılacak Devlet Katkısı Formunu (Ek-4),
- Stajyer öğrencilerin bilgilerini gösterir excel listesi,
- Stajyer öğrenciye ödeme yapıldığına dair banka dekontunu

staj yapılan her aya ait ayrı ayrı düzenleyip şirket yetkililerince imzalatarak tahakkuk ayını takip eden ayın on beşinci gününe kadar İTÜ Kariyer ve Staj Merkezi –Lisansüstü Eğitim Enstitüsü– İTÜ Ayazağa Kampüsü Maslak/SARIYER adresine ıslak imzalı evrak olarak ulaştırılması gerekmektedir. Üniversitemiz tarafından yapılan inceleme sonucunda eksik ya da hatalı belge gönderildiğinin tespit edilmesi durumunda ödeme yapılamayacaktır.

STAJIMI YÜZYÜZE YAPACAĞIM: EVET
STAJIMI ÇEVRİMİÇİ YAPACAĞIM (İSTİSNAİ): HAYIR

Öğrencinin Adı Soyadı	Staj Yapılacak Firma Adı	
Yiğit Alp Bilgin	INDEKS Bilgisayar A.Ş.	ITU-KKTC Fakültesi Dekanlığı
Tarih 10.07.2023	Tarih 10.07.2023	Tarih
İmza	KAŞE/İmza	KAŞE/İmza

INDEKS BİLGİSAYAR SİSTEMLERİ MÜHENDİSLİK SAN. TİC. A.Ş.
Ayazağa Mah. Mimar Sinan Sok. D Blk.
No:21 D: 1 Sarıyer 34485 İstanbul
Büyük Mükellefler V.D: 4780028965
Sicil No: 256376

Prof. Dr. Burak Berk ÜSTÜNDAĞ

E K - 2
Staj Başlangıç Formu
(İşyeri tarafından doldurulacaktır)

ÖĞRENCİNİN

T.C. Kimlik No	:	29195388588
Adı Soyadı	:	Yiğit Alp Bilgin
Öğrenci No	:	820210312
Bölümü	:	Bilgisayar Mühendisliği (KKTC)
Cep Tel ve E-posta Adresi	:	90-505-2221450- bilginy21@itu.edu.tr

STAJIN

Türü	:	Yazılım		
Başlangıç ve Bitiş Tarihleri	:	17.07.2023 - 11.08.2023		
Çalışılan Gün Sayısı	:	20		
Staj Günleri	:	Pazartesi	Salı	Çarşamba
		Perşembe	Cuma	
		Cumartesi	Pazar	

STAJ YAPILAN FİRMANIN

Unvanı	:	INDEKS Bilgisayar A.Ş.
Telefon No ve E-posta Adresi	:	+90-212-3312121 - info@index.com.tr
Staj adresi	:	Ayazağa Mahallesi Mimar Sinan Sokak No: 21 Seba Office Boulevard, D Blok Kat: 1 Ofis No: 11 Sarıyer, 34485 İstanbul
Formu Dolduran Firma Yetkilisinin Adı Soyadı	:	İRİS DEFNE BAROCAKÇI
Yetkilinin Unvanı	:	İ.K. DİREKTÖRÜ
Mühür-İmza-Kaşe	:	İNDEKS BİLGİSAYAR SİSTEMLERİ MÜHENDİSLİK SAN. TİC. A.Ş. Ayazağa Mah. Mimar Sinan Sok. D Blk. No:21 D:11 Sarıyer 34485 İstanbul Büyük Mükelleflier V.D: 4780028965 Sicil No: 256376 Tel.: 0212 331 21 21 Fax: 0212 289 95 25

Not: Bu belgenin elektronik kopyası, staj başlangıcından sonraki beş gün içerisinde öğrenci tarafından https://portal.itu.edu.tr web sitesine yüklenmelidir. Bu nedenle, belgenin staj başlangıcında imzalanarak öğrenciye ivedilikle teslim edilmesi önemle rica olunur. Belgenin basılı kopyası ise staj bitiminde, staj raporu ile birlikte öğrenci tarafından fakülteye ulaştırılmalıdır.

E K - 3

Staj Bitiş Formu

ÖĞRENCİNİN

T.C. Kimlik No	:	29195388588
Adı Soyadı	:	Yiğit Alp Bilgin
Öğrenci No	:	820210312
Bölümü	:	Bilgisayar Mühendisliği ( )
Cep Tel ve E-posta Adresi	:	90-505-2221450- bilginy21@itu.edu.tr

STAJIN

Yapılan Pratik Çalışma	Tarih		Staj yaptığı toplam iş günü sayısı	Kısım Amiri
	Başlama	Bitiş		
Orientation and SAP Lectures	17.07.2023	19.07.2023	3	Deniz Sarıbayır
ABAP Basıcs and Developement Tools	20.07.2023	24.08.2023	3	Deniz Sarıbayır
ABAP Operators	25.07.2023	26.07.2023	2	Deniz Sarıbayır
ABAP Control Structures	27.07.2023	28.07.2023	2	Deniz Sarıbayır
Employee Internal Table	31.07.2023	31.07.2023	1	Deniz Sarıbayır
Customer Data & Classes	01.08.2023	01.08.2023	1	Deniz Sarıbayır
Sales Order Enhancement	02.08.2023	04.08.2023	3	Deniz Sarıbayır
Data Migration	07.08.2023	08.08.2023	2	Deniz Sarıbayır
Sales Order Analysis	09.08.2023	10.08.2023	2	Deniz Sarıbayır
Product & Price List	11.08.2023	11.08.2023	1	Deniz Sarıbayır

STAJ YAPILAN İŞYERİNİN

Unvanı	:	INDEKS Bilgisayar A.Ş.
Telefon No ve E-posta Adresi	:	+90-212-3312121 - info@index.com.tr
Staj adresi	:	Ayazağa Mahallesi Mimar Sinan Sokak No: 21 Seba Office Boulevard, D Blok Kat: 1 Ofis No: 11 Sarıyer, 34485 İstanbul
Formu Dolduran Firma Yekilisinin Adı Soyadı	:	Deniz Sarıbayır
Yetkilinin Unvanı	:	Bilgisayar Mühendisi
Mühür-İmza-Kaşe	:	İNDEKS BİLGİSAYAR SİSTEMLERİ MÜHENDİSLİK SAN. TİC. A.Ş. Ayazağa Mah. Mimar Sinan Sok. D Blk. No:21 D:11 Sarıyer 34485 İstanbul Büyük Mükellefler V.D: 4780028965 Sicil No: 256376 Tel.: 0212 331 21 21 Fax: 0212 289 95 25

INTERNSHIP REPORT APPROVAL FORM

TABLE OF CONTENTS

1. INFORMATION ABOUT THE INSTITUTION 1
2. INTRODUCTION 2
3. DESCRIPTION AND ANALYSIS OF THE INTERNSHIP PROJECT 2
4. CONCLUSIONS 9
5. REFERENCES 9
6. APPENDIX 10

The internship report with the above given "Table Of Contents" was seen and approved for the student with student ID ..820210312... and name ....Yiğit Alp Bilgin.....

Authorized staff of the institution who filled in the form

Name and Surname: Deniz Sarıbayır

Title: Assistant Software Development Specialist

Manager-Signature-Seal: İNDEKS BİLGİSAYAR SİSTEMLERİ MÜHENDİSLİK SAN. TİC. A.Ş. Ayazağa Mah. Meydan Sok. D Blk. No:21 D:11 Sarıyer 34485 İstanbul Büyük Mükellefler V.D: 4780028965 Sicil No: 256376 Tel.: 0212 331 21 21 Fax: 0212 289 95 25

(Institutional) E-mail: deniz.saribayir@index.com.tr

(Institutional) Phone Number: +90 0532 645 3694

İSTANBUL TEKNİK ÜNİVERSİTESİ

ÖĞRENCİ STAJ ÇALIŞMA TAKVİMİ

Temmuz 2023

Pt	Sa	Ça	Pe	Cu	Ct	Pa
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31						

Ağustos 2023

Pt	Sa	Ça	Pe	Cu	Ct	Pa
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31			

* Öğrenciler işaretli günler dışında çalışmamalıdır. İşaretli günler dışında çalışıldığında meydana gelen tüm iş kazalarından stajın yapıldığı işletme ve öğrenci sorumlu olacaktır. Takvim üzerinde elle işaretleme ve değişiklik yapılamaz.

Öğrenci
Ad Soyad - İmza

Firma
Kaşe - İmza

ISTANBUL TECHNICAL UNIVERSITY
Faculty of Computer Science and Informatics

ABAP MASTERY IN SAP ENVIRONMENTS:
A SUMMER INTERNSHIP EXPERIENCE

INTERNSHIP PROGRAM REPORT

Yiğit Alp Bilgin
820210312

SUMMER / 2023

Istanbul Technical University

Faculty of Computer Science and Informatics

INTERNSHIP REPORT

Academic Year: 2022/2023
Internship Term: ☒ Summer ☐ Spring ☐ Fall (CHECK RELATED CHECKBOX)

Student Information

Name Surname: Yiğit Alp Bilgin
Student ID: 820210312
Department: COMPUTER ENGINEERING
Program: 100% English
E-Mail: bilginy21@itu.edu.tr
Mobile Phone: +90 (505) 222 1450

Pursuing a Double Major? ☐ Yes (Faculty/Department of DM: _____)
☒ No (CHECK RELATED CHECKBOX)

In the Graduation Term? ☐ Yes
☒ No

Taking a class at Summer School? ☐ Yes (Number of Courses:_____)
☒ No (CHECK RELATED CHECKBOX)

Institution Information

Company Name: INDEX Computer Corporation
Department: Information Technologies
Web Address: www.index.com.tr
PostalAddress: Ayazağa Mahallesi, Mimar Sinan Sokak No:21 Seba Office Boulevard, D Blok Kat: 1 Bölüm No: 11 Sarıyer, 34485 İstanbul, Turkey

Authorized Person Information

Department:	Information Technologies
Title:	Assistant Software Development Specialist
Name Surname:	Deniz Sarıbayır
Corporate E-Mail:	deniz.saribayir@index.com.tr
Corporate Phone:	+90 0532 645 3694

Internship Work Information

Internship Location:	☒ Turkey ☐ Abroad (CHECK RELATED CHECKBOX)
Internship Start Date:	17.07.2023
Internship End Date:	11.08.2023
Number of Days Worked:	20
During your internship, did you have insurance?	☒ Yes, I was insured by İTÜ. ☐ Yes, I was insured by institution. ☐ No, I did my internship abroad.

Table of Contents

1 INFORMATION ABOUT THE INSTITUTION **1**

2 INTRODUCTION **2**

3 DESCRIPTION AND ANALYSIS OF THE INTERNSHIP PROJECT **2**

3.1 Creating a Report and Defining Internal Tables and Structures 2

3.2 Fetching Data from Existing SAP Data 2

3.3 Implement Functionality 4

3.4 Creating a Web Dynpro ABAP Application 8

4 CONCLUSIONS **9**

5 REFERENCES **9**

6 APPENDIX **10**

1 INFORMATION ABOUT THE INSTITUTION

- Index Computer
- Ayazağa Mahallesi, Mimar Sinan Sokak No:21

  Seba Office Boulevard, D Blok Kat: 1

  Bölüm No: 11 Sarıyer, 34485 İstanbul, Turkey

  Telephone: +90 (212) 331 21 21

  Fax: +90 (212) 332 09 29 [1]
- Index Group, founded in 1987 by Erol Bilecik [2], took off in 1989 with the dream of becoming Turkey's ultimate tech hub [3]. Known as one of the top IT distributors in Turkey, Index AŞ, one of the five sub-companies of Index Group, specializes in information and communication technologies. They've held the top spot in the industry for the last 23 years, thanks to their skilled team, strong relationships with major global tech brands, a wide range of products, reliable logistics, and solid financial setup. Index AŞ delivers products from big names like Apple, HP, Lenovo, and many more to all corners of Turkey through their extensive network of 8000 business partners [4].
- Index Group Sub-Companies [6]:
  - Index AŞ
  - Datagate AŞ
  - Despec AŞ
  - Netex AŞ
  - Teklos AŞ
- Product Groups [5]:
  - Computer/Tablets
  - Smartphones
  - Printers
  - Wearable Technologies
  - OEM Technologies
  - Data Storage
  - Consumer Electronics
- In the Information Technologies and Software Department, the engineers are briefly responsible for organizing and managing company data flow. The department has two groups where one handles the web development orders for the customers. In contrast, I was operating in the other group solving the business-to-business issues of the company's customers via SAP software. Moreover, in the SAP faction, I had the authorization to add or remove new custom functionalities to customers' SAP modules using the ABAP coding language.

2 INTRODUCTION

In my Internship project, I aimed to learn more about the SAP [7]program/ABAP language and develop a new functionality menu inside the SAP application using my newly learned skills, where the user can review and print different sale-related data.

My project consisted of:

- Creating a Report and Defining Internal Tables and Structures
- Fetching Data from Existing SAP Data
- Implementing Functionality
- Creating a Web Dynpro ABAP Application

3 DESCRIPTION AND ANALYSIS OF THE INTERNSHIP PROJECT

3.1 Creating a Report and Defining Internal Tables and Structures

In my SAP GUI, I navigated to the ABAP Editor using transaction SE38 to create a new report program, which I named **ZCONSUMER-INFO** Figure 1

Within this program, I defined essential internal tables and structures to effectively manage consumer, product, and sales data.(Figure 2). Specifically, **ty-consumer**, **ty-product**, and **ty-sales** represented the structures outlining the distinct components of consumer, product, and sales information respectively. I utilized these structures to establish internal tables - **lt-consumers**, **lt-products**, and **lt-sales** - each serving as a container capable of holding multiple consumer, product, and sales data records.(Figure 3)

These internal tables were crucial as they allowed me to store and manipulate data directly within my ABAP program. I populated these tables by fetching data from various sources, including databases. The acces for the database and necessary sdsources provided me by my supervisor. Once the data resided within these internal structures, I could execute a multitude of operations. This included tasks like data filtering, sorting, and presenting the information in diverse formats tailored to the unique requirements of my project. These steps enabled seamless handling and analysis of consumer, product, and sales data within the project framework.(Figure 4).

3.2 Fetching Data from Existing SAP Data

In this section, I retrieved data from existing SAP records. First, I constructed the **SELECT** query where I specified the fields I needed from the SAP table and the table from which I wanted to extract data. If necessary, I added conditions using the **WHERE** clause:

- **Fields Selection:** I specified the fields (**field1, field2, etc.**) I wanted to fetch from the SAP table.
- **SAP Table Name:** I provided the name of the SAP table I wanted to extract data from, labeled as 'your-sap-table.'
- **Internal Table Setu**p: I defined an internal table ('**lt-data**') where the fetched data would be stored.

- **Optional Filtering:** If needed, I could include an optional WHERE clause ('your-condition') to filter the data based on specific conditions.

To fetch data from my previously defined internal tables like **lt-consumers, lt-products,** and **lt-sales**, along with their corresponding structures **ty-consumer, ty-product,** and **ty-sales**, I formulated **SELECT** statements for each internal table:

1. For lt-consumers:

    - **SELECT Statement:** I specified the fields to fetch: **consumer-id, consumer-name**, and **credit-limit**.
    - **FROM TABLE @lt-consumers:** I indicated that data was retrieved from the internal table **lt-consumers.**
    - **INTO TABLE @DATA(lt-result-consumers):** I ensured that the selected data was stored in a new internal table, lt-result-consumers. (Figure 5)

2. For lt-products:

    - **SELECT Statement:** I defined the fields to extract: product-id, product-name, and product-price.
    - **FROM TABLE @lt-products:** I mentioned that data was sourced from the internal table lt-products.
    - **INTO TABLE @DATA(lt-result-products):** I confirmed that the extracted data was placed in a new internal table, lt-result-products (Figure 6;).

3. For lt-sales:

    - **SELECT Statement:** I specified the fields to be retrieved: product-id, order-date, customer-name, quantity, and total-revenue.
    - **FROM TABLE @lt-sales:** I denoted that data was fetched from the internal table lt-sales.
    - **INTO TABLE @DATA(lt-result-sales):** I ensured that the fetched data was stored in a fresh internal table, lt-result-sales Figure 7.

In these SELECT Statements:

- **SELECT** was used for field selection.
- **FROM TABLE @lt-internal-table** pointed to the source internal table.
- **INTO TABLE @DATA(new-internal-table)** signified where the extracted data would be stored.

Upon execution, **lt-result-consumers**, **lt-result-products**, and **lt-result-sales** held the chosen data from **lt-consumers**, **lt-products**, and **lt-sales** respectively. This allowed me to further process the data within my project. It was important to replace placeholders like field names, table names, and conditions with the actual ones matching my project requirements and ensure the data types aligned between the internal table (**lt-data**) and the SAP table fields.

3.3 Implement Functionality

In this step of my project, I wrote the necessary code for the different functionalities.

1. **Product list SAP Web Dynpro code**

   The first menu I added to the project was the Products List functionality for the stock, printing the requested data for the user by pulling the data from the pre-fetched data.
   First, the objective was to populate a context node with product data sourced from a custom class. Here's a breakdown of what I did: (Figure 8)

   (a) Data Declarations:

   - **lo-nd-product-list:** I declared this variable to hold a reference to a context node named 'ProductList,' an important data storage item in SAP Web Dynpro.
   - **lo-el-product-list:** Another variable was set up to hold a reference to a context element inside the 'ProductList' node. Context elements serve as containers for individual records within a node.
   - **lt-products:** An internal table, lt-products, was declared to store instances of a custom class, zcl-product. This table was designated to hold the fetched product data.

   (b) Obtaining a Reference to the Context Node:

   - **lo-nd-product-list = wd-context -> get-child-node(name = 'ProductList'):** I retrieved a reference to the context node named 'ProductList.' This node had to be pre-defined in the SAP Web Dynpro component's context.

   (c) Fetching Product Data:

   - **lt-products = zcl-product => get-product-list( ):** I invoked a static method, **get-product-list**, from the class **zcl-product** to fetch a list of product data. The fetched data was stored in the internal table, **lt-products**.

   (d) Iterating Through Product Data and Populating Context Node:

   - **LOOP AT lt-products INTO DATA(ls-product):** I employed a loop to iterate through the product data stored in **lt-products**.
   - **lo-el-product-list = lo-nd-product-list => create-element( ):** Within the loop, a new context element was dynamically created for each product.
   - **lo-el-product-list -> set-attribute(name = 'ProductId' value = ls-product-product-id):** I set the 'ProductId' attribute of the context element with the corresponding product ID from the fetched data.
   - Similar lines were used to set the **'ProductName'** and **'ProductPrice'** attributes.

   In summary, this code allowed the retrieval of a product data list from a custom class (zcl-product). Through the process, the attributes of each product (ID, name, and price) were added to a context node named 'ProductList.' This context node

was subsequently utilized in the user interface, potentially to display the product information within tables or other UI elements in the SAP Web Dynpro application (Figure 9).

2. **Customer Related Data Display Code**

   In this part, the functionality of this menu was to print the Customer Related data, pulling the data from the pre-fetched data (Figure 10).

   First, I developed an ABAP class called **zcl-customer**, which had the purpose to handle customer data efficiently. Steps were:

   Class Definition (CLASS zcl-customer DEFINITION).

   (a) Attributes:

   - **customer-id TYPE i**: This attribute held the customer's ID, represented as an integer in ABAP.
   - **customer-name TYPE string:** It stored the customer's name as a string.
   - **credit-limit TYPE f:** This attribute signified the customer's credit limit, stored as a floating-point number in ABAP.

   (b) Methods:

   - **constructor:** This method served as the class constructor, initializing the object during its creation. It took three input parameters: customer ID (**i-id**), customer name (**i-name**), and credit limit (**i-limit**), assigning these values to the corresponding attributes.
   - **get-id:** This method retrieved and returned the customer's ID (**customer-id**).
   - **get-name:** Similarly, this method fetched and returned the customer's name (**customer-name**).
   - **get-credit-limit:** It retrieved and returned the customer's credit limit (**credit-limit**).
   - **set-credit-limit:** This method facilitated the modification of the credit-limit attribute. By taking an input parameter (**i-limit**), it updated the credit-limit attribute with the provided value.

   Class Implementation (CLASS zcl-customer IMPLEMENTATION).

   (a) Constructor Implementation (**METHOD constructor**).

   - Whenever a new object of the class was created, this method was automatically invoked.
   - It initialized the customer-id, customer-name, and credit-limit attributes with the values passed as parameters.

   (b) Getter Methods (**METHOD get-id, get-name, get-credit-limit**).

   - These methods straightforwardly retrieved and returned the values of the corresponding attributes.

   (c) Setter Method (**METHOD set-credit-limit**).

- This method helped the process of modification of the credit-limit attribute. It set the credit-limit attribute to the value passed as an input parameter.

In essence, this class served as a outline for managing customer data. It could create customer objects with specified ID, name, and credit limit. Additionally, it provided methods to retrieve the customer's ID, name, and credit limit, along with a function to update the credit limit. Such classes were pivotal in object-oriented programming as they encapsulated both data and behaviors pertinent to a specific entity, in this case, a customer (Figure 11).

1, **Sales Order "User Exit" Code**

In this part, the functionality of this menu was to determine if the requested product numbers exceeds the stocks or not &Dgespc/0'.

I implemented the code to validate if the quantity of a product ordered in a sales transaction surpassed the available stock. Here's how I executed the process:

&' Variable Declarations:

- **lv-available-stock TYPE i:** I declared this variable to store the available stock quantity of the product, specified as an integer.
- **lv-ordered-quantity TYPE i:** Another variable, set as an integer type, was utilized to hold the quantity of the product ordered.
- **lv-product-id TYPE string:** This variable was designated to contain the product ID of the item within the sales order, defined as a string.

&`' Data Assignment:

- **lv-product-id = <product-id>:** Here, I assigned a specific product ID to lv-product-id, replacing the placeholder <product-id> with the actual product ID from the sales order.
- **lv-ordered-quantity = <ordered-quantity>:** Similarly, the variable lv-ordered-quantity was assigned the real quantity of the item from the sales order, replacing <ordered-quantity>.

&a' Function Call:

- **CALL-FUNCTION 'Z-GET-AVAILABLE-STOCK':** I initiated a call to a custom function module named **Z-GET-AVAILABLE-STOCK**, aiming to retrieve the available stock quantity based on the provided **lv-product-id.**
- **EXPORTING product-id = lv-product-id:** The function received **lv-product-id** as an input parameter.
- **IMPORTING available-stock = lv-available-stock:** The function returned the available stock quantity, which I stored in **lv-available-stock.**
- EXCEPTIONS: To handle potential exceptions, I set up this block.
  - **data-not-found = 1:** This exception was programmed to trigger if data (available stock) corresponding to the provided product ID was not found.

- OTHERS = 2: Catching all other exceptions that might occur during the function execution.

(d) Result Checking:

- **IF sy-subrc <> 0:** If an exception occurred during the function call (indicated by sy-subrc not being 0), implying data was not found, an error message was generated.
- **ELSE:** In the absence of exceptions, signifying data was retrieved successfully.
- **IF lv-ordered-quantity > lv-available-stock:** I compared the ordered quantity with the available stock quantity.
- **MESSAGE 'Ordered quantity exceeds available stock.' TYPE 'E':** If the ordered quantity exceeded the available stock, the system generated an error message.

In summary, this code I developed, verified if the product quantity ordered in a sales transaction exceeded the available stock. It ensured error messages were generated in cases where data was not found or if the ordered quantity surpassed the available stock quantit (Figure 13).

4. **Sales Order Analysis" SAP ALV Code**

In this menu, the aim was to implement functionality to analyze sales orders within a specific timeframe. (Figure 14):

(a) Defining Data Structures:

I designed a structure, **ty-sales-order**, outlining the format of our sales order data. This structure contained vital details like order ID, order date, customer name, product name, quantity, and total revenue.

(b) Managing Data with Internal Tables:

To handle the fetched sales order data, I utilized an internal table named **lt-sales-orders**. This table adhered to the structure **ty-sales-order**, ensuring organized data storage.

(c) Working with ALV:

I declared two crucial variables. **lo-alv** represented our ABAP List Viewer (ALV) grid, while **lo-alv-functions** held references to ALV functions for user interactions.

(d) User-Friendly Selection Screen:

For user convenience, I integrated selection screen parameters: **p-start-date** and **p-end-date**. These parameters enabled users to specify the desired sales order timeframe, with default values set to '2023-01-01' and '2023-12-31' respectively.

(e) Efficient Data Retrieval:

I implemented a method **get-sales-order-data** from the class **zcl-sales-order-report**. This method expertly fetched sales order data from the database, adhering to the specified date range. The retrieved information seamlessly filled the **lt-sales-orders** internal table.

(f) Streamlining ALV Creation:

Utilizing the class **cl-salv-table**, I efficiently created the ALV grid. I configured it not to display immediately **(list-display = abap-false)**, ensuring our data was pristine. The ALV grid's data source was **lt-sales-orders**, holding our refined sales order data.

(g) Handling Exceptions:

I implemented error handling, particularly **CATCH cx-salv-msg**. This catch block ensured we gracefully managed exceptions, enhancing the robustness of our application.

(h) Tailoring ALV Functionality:

To enrich user experience, I enabled all ALV functions through **lo-alv -> get-functions( ) -> set-all(abap-true)**. Additionally, I set the ALV grid's layout as default using **lo-alv -> get-layout( ) -> set-default(abap-true)**, ensuring a cohesive and intuitive user interface.

(i) Showcasing the Results:

In the culmination of my efforts, I triggered the ALV grid's display with **lo-alv -> display( )**. This action unveiled our meticulously processed sales order data within the specified date range, presenting it to users with enhanced clarity and functionality (Figure 15).

3.4 Creating a Web Dynpro ABAP Application

1. Creating the Web Dynpro Component:
   - I initiated the process by accessing transaction SE80.
   - From the repository browser, I selected Web Dynpro.
   - Subsequently, I right-clicked on my package and created a new Web Dynpro Component, naming it ZCONSUMER-UI.
2. Building the View:
   - Inside my Web Dynpro component, I crafted a new View.
   - In this view, I designed the layout and various UI components, such as input fields for Product ID, Customer ID, Quantity, buttons for different actions, and an area to display results.
3. Defining Context Nodes:
   - I defined specific context nodes to store the data I intended to showcase and manage in the user interface. For instance, nodes were created for Product, Customer, and Sales data.
4. Linking UI Elements with Context Nodes:
   - I established connections between the UI elements in my view and the respective context nodes. This step ensured that the UI elements were synchronized with the data structures in my ABAP code.
5. Implementing Controller Logic:
   - I developed the underlying logic responsible for retrieving data from SAP tables based on user inputs. This logic closely resembled the operations in my ABAP report.

- Additionally, I handled user interactions such as button clicks. For instance, when the user clicked ”Print Product List,” the application fetched the relevant product data and displayed it in the UI.

6. Conducting Testing:

    - To ensure everything was functioning as intended, I executed my Web Dynpro component using transactions SE80 or SE71.
    - During testing, I input various data sets to verify that the UI responded correctly, displaying the expected data according to the user’s inputs.

To sum it up, this section's purpose was to develope an interface for the customer. I built and tested the Web Dynpro component, ensuring a seamless user experience and accurate data representation.

4 CONCLUSIONS

The most crucial aspect of my 20-day long internship was learning about the hardships of being a software specialist in a very institutional company that is one of the leaders in its field. Now that I have seen the world of business, I can adequately prepare myself for my future career and canalize myself to the most suitable area in computer science.

One of the other benefits of this internship was learning about SAP and ABAP—one of the essential softwares in the business world and its core software language. In my short journey, I tried to learn as much as possible about these topics, which I had never heard of before, from firsthand engineers who were experienced in their fields. Moreover, since SAP is not a software on sale for third-party consumers, I could use and experience a program only available to a few engineers.

Furthermore, meeting with engineers who were experienced in their field helped me to learn more about the sector and made me feel lucky. I had the chance to prepare brief hands-on example projects for every topic I have learned and improved my newly discovered ABAP skills.

To conclude, after these 20 days of learning experience, I have learned a lot of new concepts and improved myself by developing a functional project. I have become more prepared for future business opportunities, and I plan to work with my colleagues again on future projects.

5 REFERENCES

[1] Company Contact Info, `https://www.index.com.tr/en/contact/contact-info`

[2] About Company, `https://www.index.com.tr/en/corporate/about-us`

[3] Corporate History, `https://www.index.com.tr/en/corporate/history`

[4] Corporate Vision, `https://www.index.com.tr/en/corporate/vision-mission`

[5] Corporate Management, `https://www.index.com.tr/en/corporate/management`

[6] Sub-companies, `https://www.index.com.tr/en/brands`

[7] About SAP, `https://www.sap.com/about.html`

.

.

6 APPENDIX

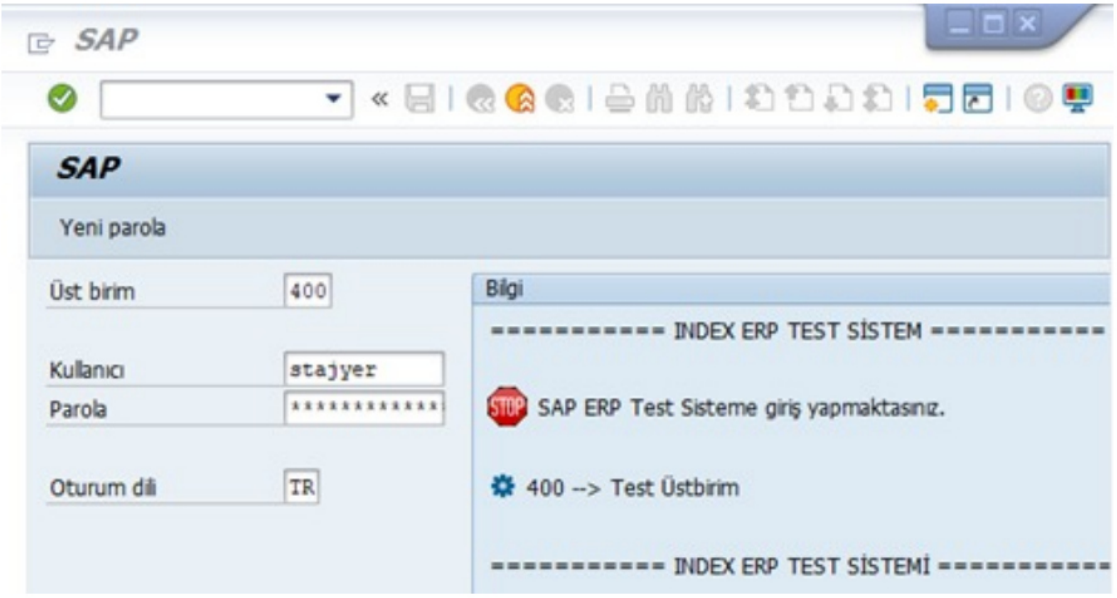

Figure 1: SAP Login

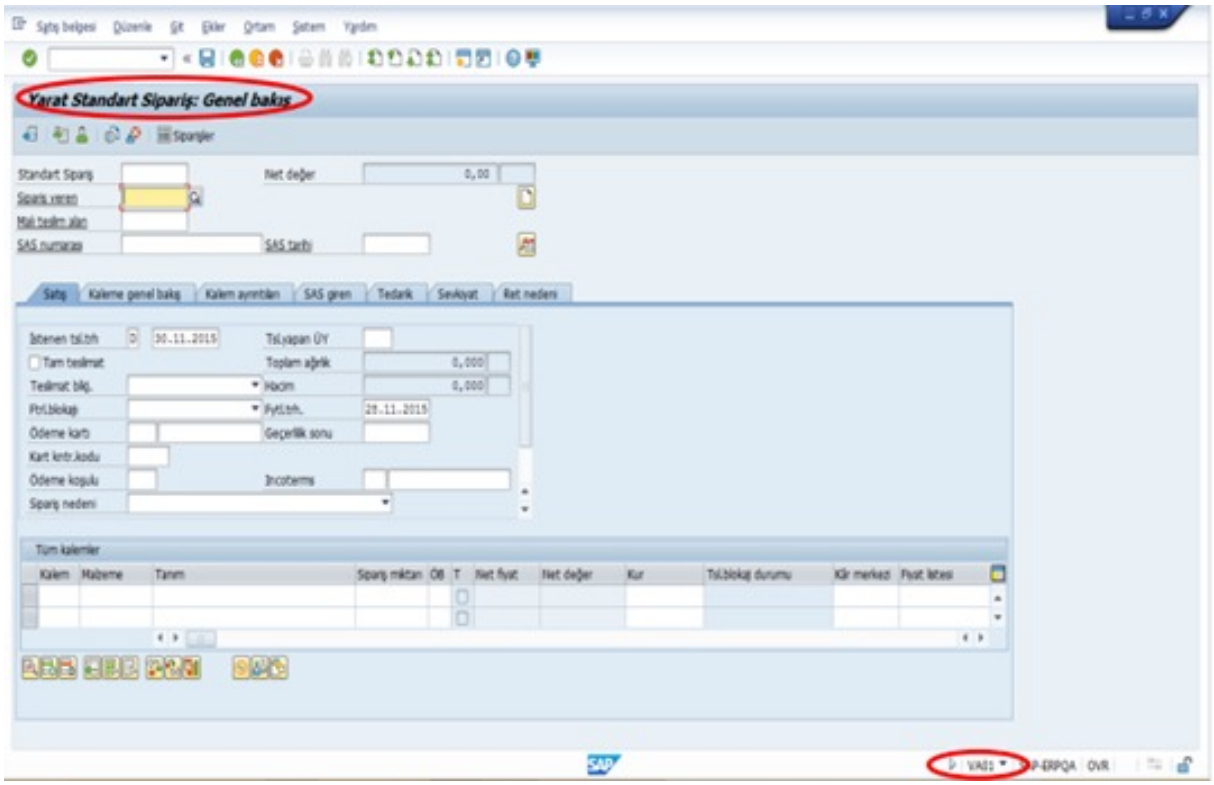

Figure 2: SAP Creation Menu

```
TYPES: BEGIN OF ty_consumer,
         consumer_id TYPE i,
         consumer_name TYPE string,
         credit_limit TYPE p LENGTH 10 DECIMALS 2,
       END OF ty_consumer.

TYPES: BEGIN OF ty_product,
         product_id TYPE string,
         product_name TYPE string,
         product_price TYPE p LENGTH 10 DECIMALS 2,
       END OF ty_product.

TYPES: BEGIN OF ty_sales,
         product_id TYPE string,
         order_date TYPE d,
         customer_name TYPE string,
         quantity TYPE i,
         total_revenue TYPE p LENGTH 10 DECIMALS 2,
       END OF ty_sales.

```

Figure 3: Defining Structure Code

```
DATA: lt_consumers TYPE TABLE OF ty_consumer,
      lt_products TYPE TABLE OF ty_product,
      lt_sales TYPE TABLE OF ty_sales.

```

Figure 4: Table Type

```
SELECT consumer_id, consumer_name, credit_limit
FROM TABLE @lt_consumers
INTO TABLE @DATA(lt_result_consumers).
```

Figure 5: Select for lt-consumers

```
1. SELECT product_id, product_name, product_price
2. FROM TABLE @lt_products
3. INTO TABLE @DATA(lt_result_products).
```

Figure 6: Select for lt-products

```
1. SELECT product_id, order_date, customer_name, quantity, total_revenue
2. FROM TABLE @lt_sales
3. INTO TABLE @DATA(lt_result_sales).
```

Figure 7: Select for lt-sales

```
DATA: lo_nd_product_list TYPE REF TO if_wd_context_node,
      lo_el_product_list TYPE REF TO if_wd_context_element,
      lt_products TYPE TABLE OF zcl_product.

* Get a reference to the context node
lo_nd_product_list = wd_context->get_child_node( name = 'ProductList' ).

* Fetch the list of products (assuming zcl_product is a custom class representing a
product)
lt_products = zcl_product=>get_product_list( ).

* Loop through the product data and add it to the context node
LOOP AT lt_products INTO DATA(ls_product).
  lo_el_product_list = lo_nd_product_list->create_element( ).
  lo_el_product_list->set_attribute( name = 'ProductId' value = ls_product-product_id ).
  lo_el_product_list->set_attribute( name = 'ProductName' value = ls_product-product_name
).
  lo_el_product_list->set_attribute( name = 'ProductPrice' value = ls_product-
product_price ).


ENDLOOP.
```

Figure 8: Product List Code Snippet

```
ProductList
|-- Product (Element)
|    |-- ProductId: 1001
|    |-- ProductName: Product A
|    |-- ProductPrice: 50.00
|    |
|-- Product (Element)
|    |-- ProductId: 1002
|    |-- ProductName: Product B
|    |-- ProductPrice: 75.00
|    |
|-- Product (Element)
|    |-- ProductId: 1003
|    |-- ProductName: Product C
|    |-- ProductPrice: 120.00
```

Figure 9: Product List Output Example

```
CLASS zcl_customer DEFINITION.
  PUBLIC SECTION.
    DATA: customer_id TYPE i,
          customer_name TYPE string,
          credit_limit TYPE f.

    METHODS: constructor IMPORTING i_id TYPE i i_name TYPE string i_limit TYPE f,
             get_id RETURNING VALUE(r_id) TYPE i,
             get_name RETURNING VALUE(r_name) TYPE string,
             get_credit_limit RETURNING VALUE(r_limit) TYPE f,
             set_credit_limit IMPORTING i_limit TYPE f.
ENDCLASS.

CLASS zcl_customer IMPLEMENTATION.
  METHOD constructor.
    customer_id = i_id.
    customer_name = i_name.
    credit_limit = i_limit.
  ENDMETHOD.

  METHOD get_id.
    r_id = customer_id.
  ENDMETHOD.

  METHOD get_name.
    r_name = customer_name.
  ENDMETHOD.

  METHOD get_credit_limit.
    r_limit = credit_limit.
  ENDMETHOD.

  METHOD set_credit_limit.
    credit_limit = i_limit.
  ENDMETHOD.
ENDCLASS.
```

Figure 10: Customer Info Code Snippet

```
Customer ID: 1001
Customer Name: Burak Simsek
Credit Limit: 5000.00

Updated Credit Limit: 7000.00
```

Figure 11: Customer Info Output Example

```
DATA: lv_available_stock TYPE i,
      lv_ordered_quantity TYPE i,
      lv_product_id TYPE string.

lv_product_id = <product_id>. "Replace <product_id> with the actual product ID of the
sales order item.
lv_ordered_quantity = <ordered_quantity>. "Replace <ordered_quantity> with the actual
quantity of the sales order item.

" Logic to fetch available stock quantity for the product from the inventory system.
" This could be a custom function module or a database query.
CALL FUNCTION 'Z_GET_AVAILABLE_STOCK'
  EXPORTING
    product_id = lv_product_id
  IMPORTING
    available_stock = lv_available_stock
  EXCEPTIONS
    data_not_found = 1
    OTHERS = 2.

IF sy-subrc <> 0.
  MESSAGE 'Error occurred while fetching available stock quantity.' TYPE 'E'.
ELSE.
  IF lv_ordered_quantity > lv_available_stock.
    MESSAGE 'Ordered quantity exceeds available stock.' TYPE 'E'.
  ENDIF.
ENDIF.
```

Figure 12: Stock Quantity Code Snippet

```
Ordered quantity exceeds available stock.
```

Figure 13: Stock Quantity Output Example

```
REPORT z_sales_order_analysis.

* Types declarations for sales order data
TYPES: BEGIN OF ty_sales_order,
         order_id TYPE n LENGTH 10,
         order_date TYPE d,
         customer_name TYPE string,
         product_name TYPE string,
         quantity TYPE i,
         total_revenue TYPE p LENGTH 10 DECIMALS 2,
       END OF ty_sales_order.

* Internal table to hold sales order data
DATA: lt_sales_orders TYPE TABLE OF ty_sales_order.

* ALV declarations
DATA: lo_alv TYPE REF TO cl_salv_table,
      lo_alv_functions TYPE REF TO cl_salv_functions_list.

* Selection screen parameters
PARAMETERS: p_start_date TYPE d DEFAULT '2023-01-01',
            p_end_date TYPE d DEFAULT '2023-12-31'.

* Fetch sales order data (Replace this with your actual logic to fetch data from database)
lt_sales_orders = zcl_sales_order_report=>get_sales_order_data( p_start_date, p_end_date
).

* Create ALV object
TRY.
    cl_salv_table=>factory(
      EXPORTING
        list_display = abap_false
      IMPORTING
        r_salv_table = lo_alv
      CHANGING
        t_table      = lt_sales_orders ).
CATCH cx_salv_msg.
ENDTRY.

* Set ALV attributes
lo_alv->get_functions( )->set_all( abap_true ).
lo_alv->get_layout( )->set_default( abap_true ).

* Display ALV report
lo_alv->display( ).
```

Figure 14: Sale Info Code Snippet

order_id	order_date	customer_name	product_name	quantity	total_revenue
1001	2023-05-15	Datagate	Product X	5	250.00
1002	2023-06-10	Despec	Product Y	10	500.00
1003	2023-07-20	INDEX	Product Z	8	400.00

Figure 15: Sale Info Output Example